



NANYANG
TECHNOLOGICAL
UNIVERSITY
SINGAPORE

SC6109 – Blockchain Privacy & Scalability

Lecture 05

Assoc Prof Guo Jian
SPMS



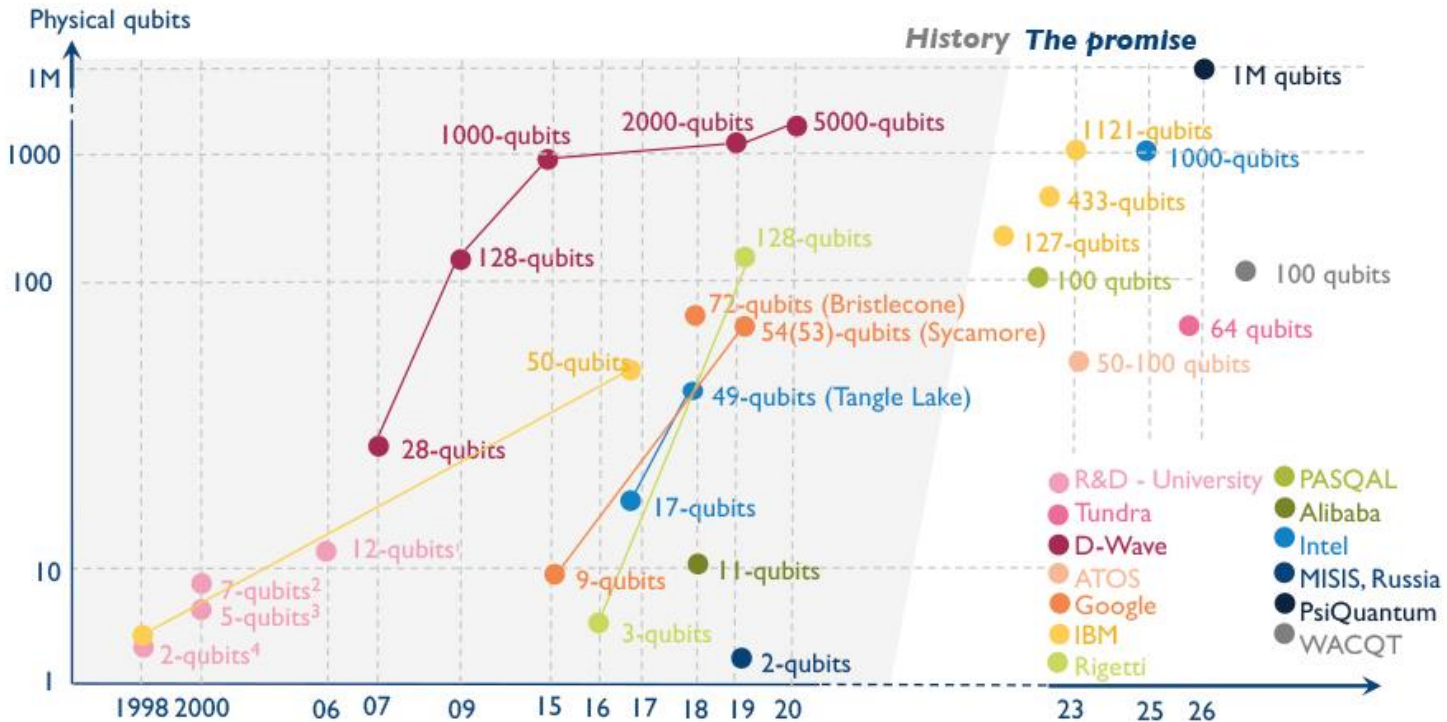
Post-Quantum Cryptography and Its Impact on Blockchain and Cryptocurrency



Development of Quantum Computers

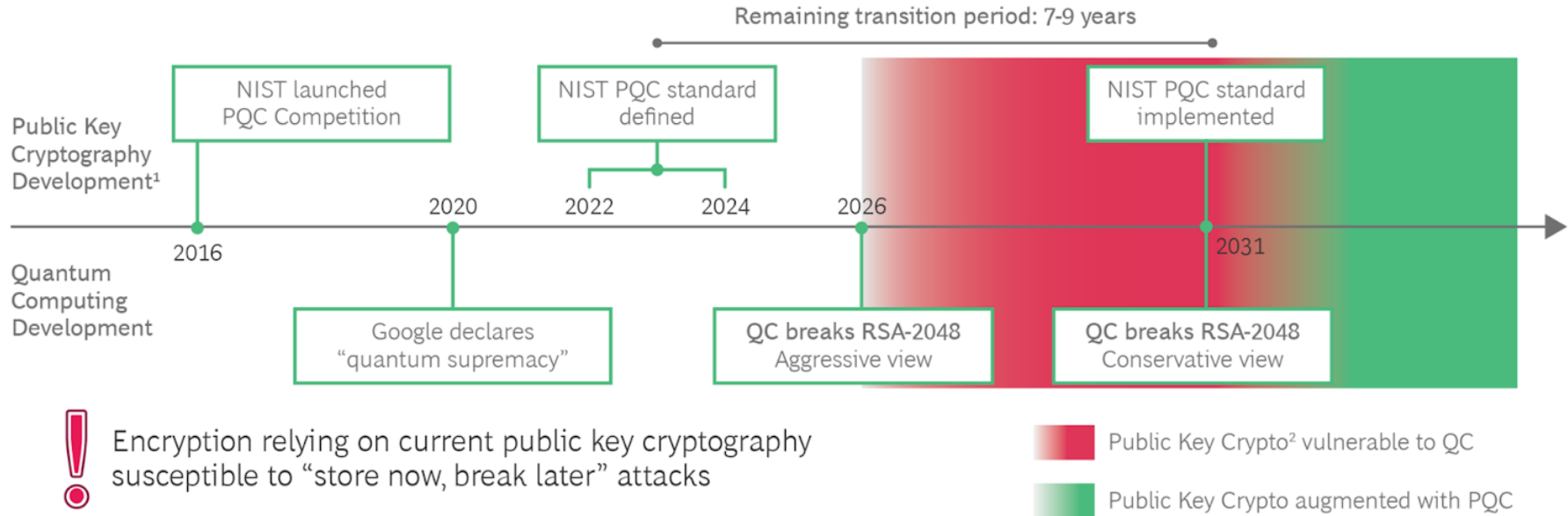


A Roadmap of Quantum Computing



source: <https://www.yolegroup.com/product/report/quantum-technologies-2021/>

Disruption of Public-Key Cryptography



Sources: NIST Post-Quantum Cryptography timeline, BCG analysis.

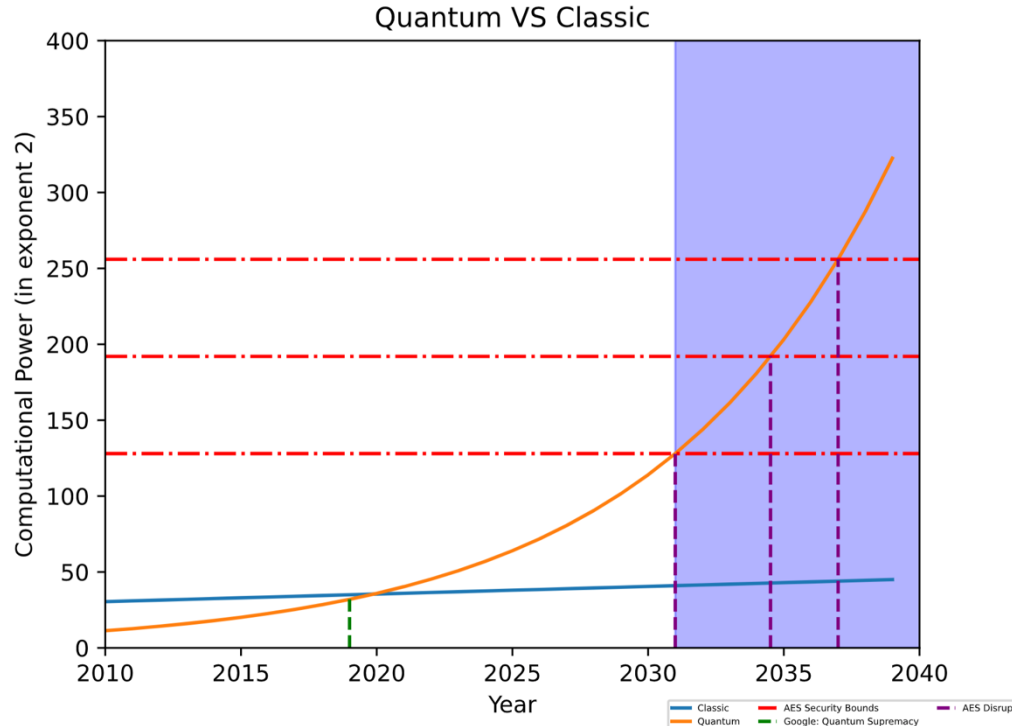
Note: PQC: Post-Quantum Cryptography. NIST: National Institute of Standards and Technology USA.

¹Based on NIST PQC timeline.

NIST Post-Quantum Cryptography (PQC) Standardization Process



Generalized Moore's Law Applied to AES



Assumptions taken:

- ✓ The growth of the qubit follows a generalized Moore's Law (*Based on current statistics, qubits doubles every 6 years*)
- ✓ The computational power of n qubits is *generally equivalent* to 2^n classical bits

All versions of AES will be broken in 15 years !

Meet The Quantum Adversaries

- Grover's Search

- Quadratic speedup on brute force search of preimage: $2^n \rightarrow 2^{n/2}$
- Speedup on collision finding: $2^{n/2} \rightarrow 2^{n/3}$

Example: for hash function standard AES-MMO, a quantum computer can find a preimage in 2^{64} computations instead of 2^{128} and a collision in 2^{43} computations instead of 2^{64}

- Simon's Period Finding

- Linearized time for finding period behaviors, i.e., if $f(x) = f(x+s)$ for all x , then finding the period s is **easy** by quantum computer
- Break MAC standards (CBC-MAC, PMAC, GMAC, etc.)

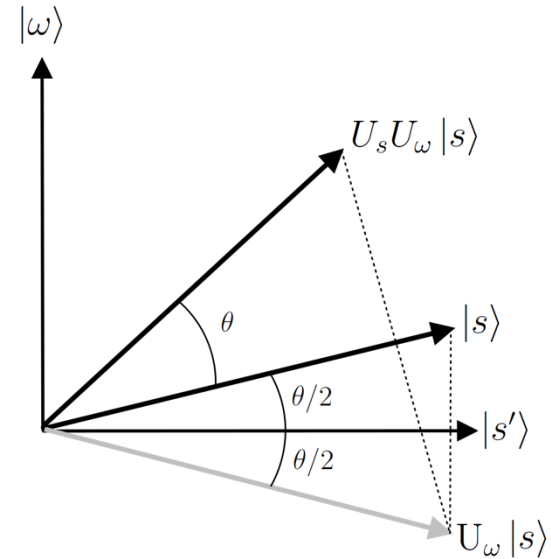
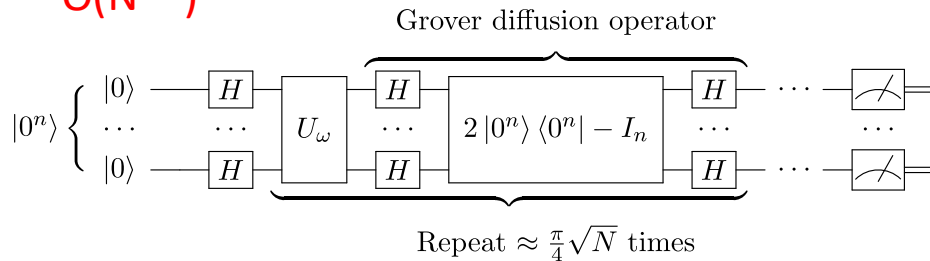
$\sim O(\text{poly}(n))$
v.s. $2^{n/2}$ in classical

- Quantum Fourier Transform

- An efficient quantum alternative for Fast Fourier Transform
- Accelerate key recovery attacks

Example: Grover's Algorithm

- Unstructured Search Problem
Find a qualified element in a large and **unstructured** database (*no sorting allowed*)
- Best Classic Algorithm
Brute-force traverse the database $O(N)$
- Quantum Circuit Design of Grover's Algorithm
 $O(N^{1/2})$

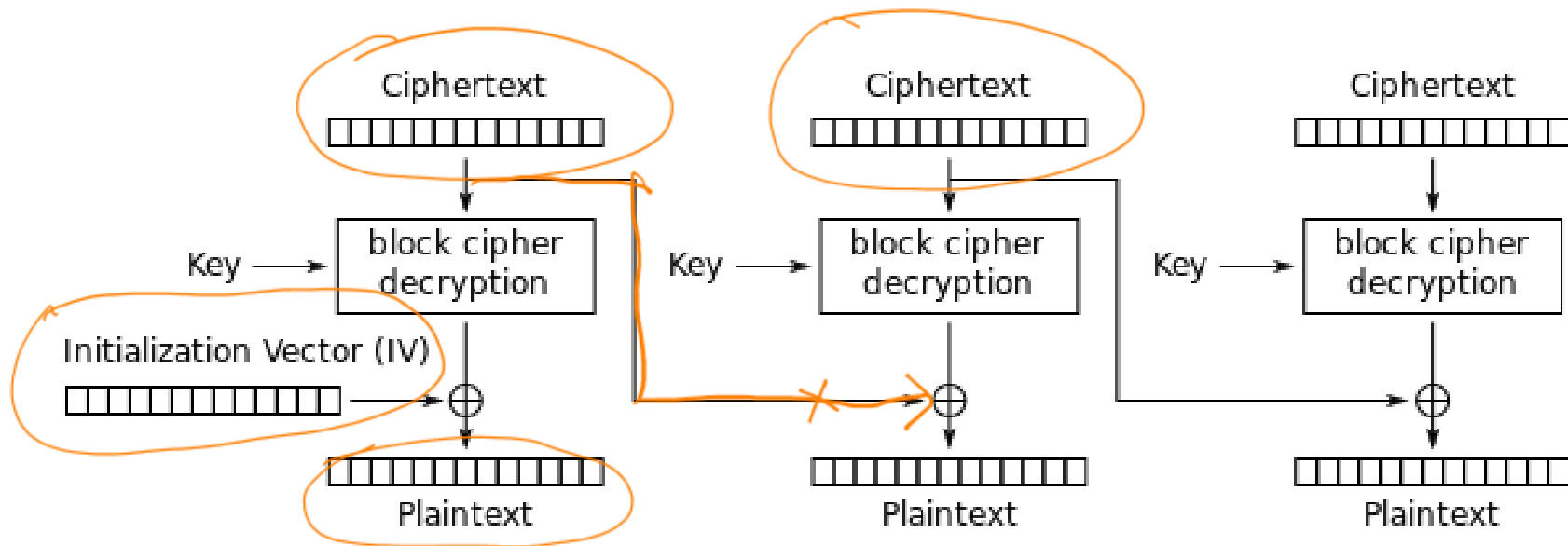


“Amplitude Amplification”

Modes of Operations of Block Ciphers

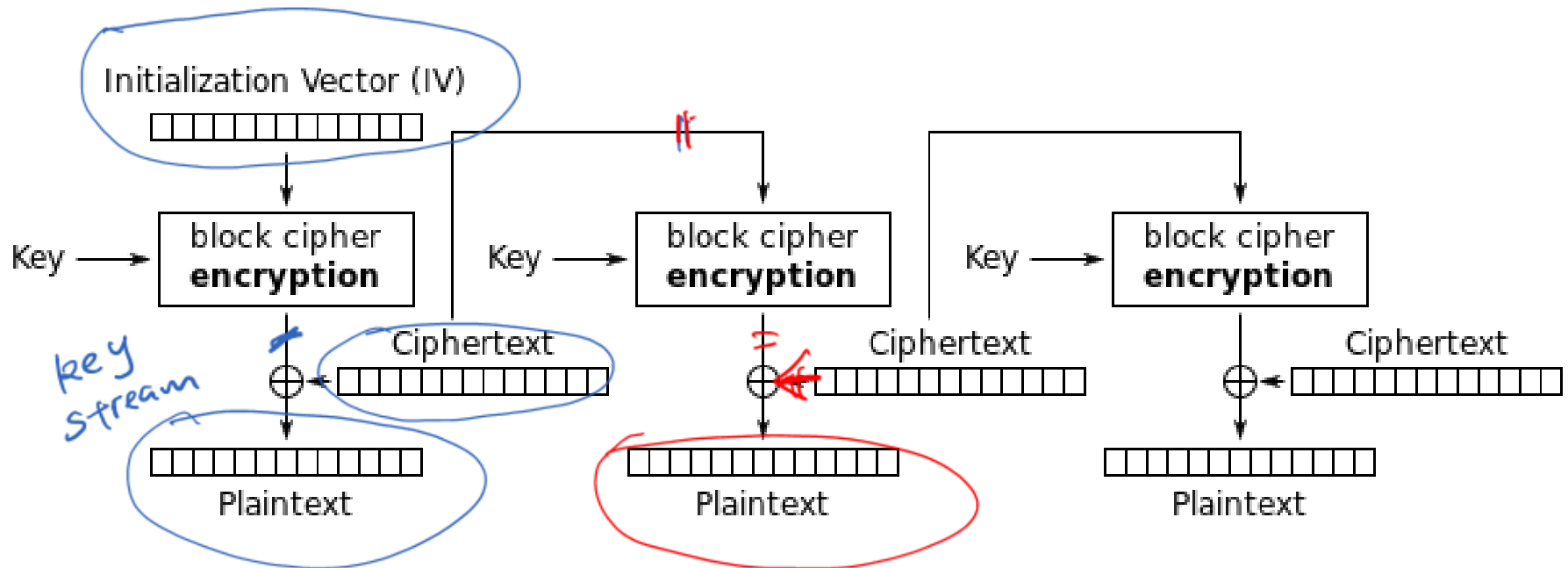
- Definition: The repetition mode a cipher adopts on single-block operations
- Common modes:
 - Electronic Codebook (ECB)
 - Cipher Block Chaining (CBC)
 - Counter (CTR)

Example: CBC Mode



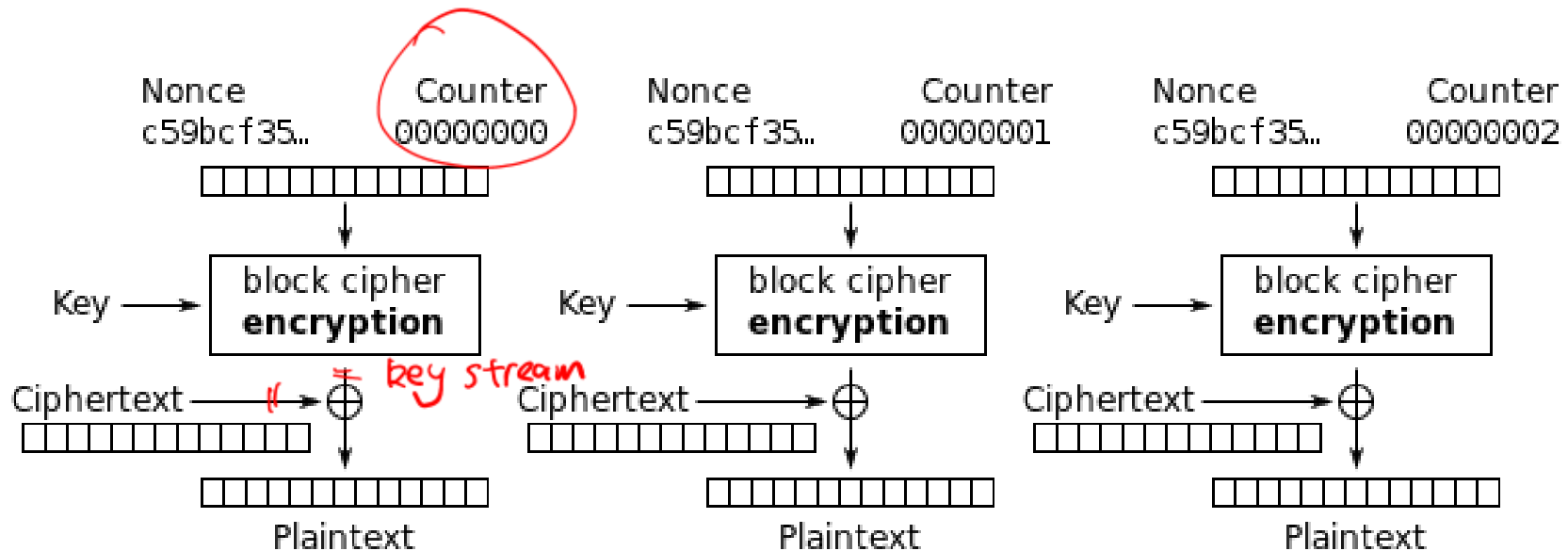
Cipher Block Chaining (CBC) mode decryption

Example: CFB Mode



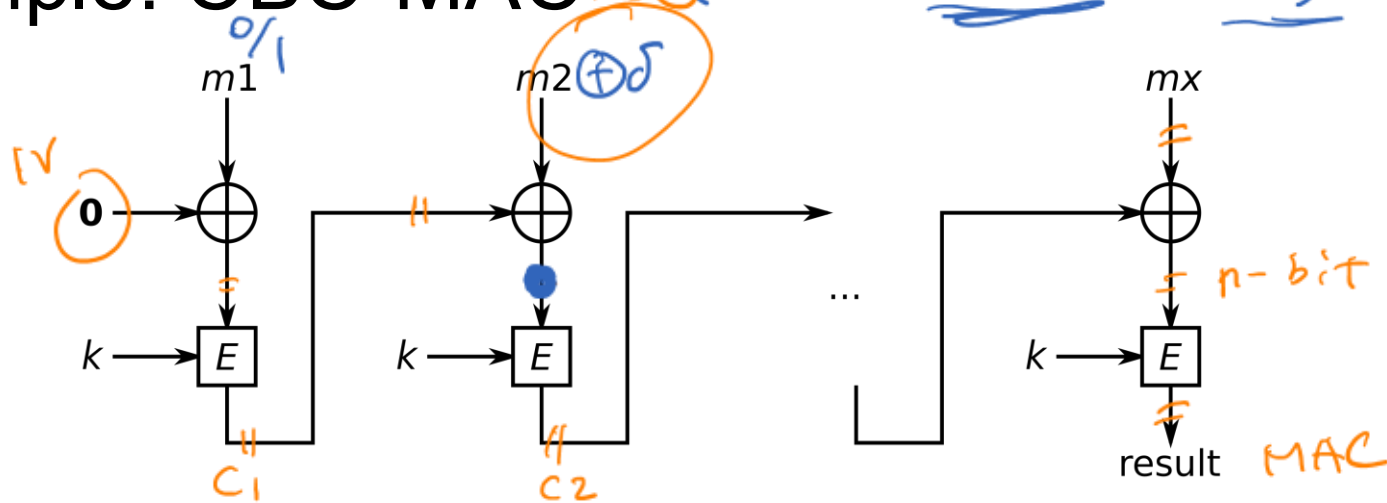
Cipher Feedback (CFB) mode decryption

Example: CTR Mode



Counter (CTR) mode decryption

Example: CBC-MAC



- Forgery Attack on MAC

Given access to a MAC oracle, **forge** a valid message/tag pair

- Classical Security of CBC-MAC Against Forgery Attacks:

If a CBC-MAC is built on a secure block cipher, forgery attacks work in $2^{n/2}$

Example: Quantum Forgery on CBC-MAC

- Period Finding Problem

Given a function f that is periodic, i.e., $f(s \oplus x) = f(x)$ for all x , to find the period s

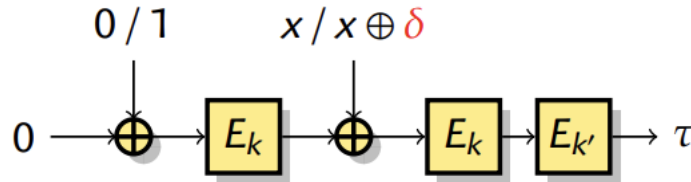
Best Classic Algorithm: $O(N^{1/2})$

- A Glimpse on Quantum Forgery Based on Period Finding

If a δ is found, then based on a message $m = b | x | \dots$, and its tag τ ,

We can forge a message $m' = b | x \oplus \delta | \dots$ and with the same tag τ

By Simon's Algorithm $O(\log(N))$



near time,

Quantum Threats To Security Guarantees

- Red Zone

Completely broken under quantum attacks, quantum-resistant alternatives are in urgent need

- Modes of operations of block ciphers (CBC-MAC, PMAC, GMAC, etc.)
- Collision of Hash with digest size < 160 bits, AES-MMO

- Yellow Zone

Quantum computers are able to attack a reduced-round symmetric-key primitive, and the general security bound has been reduced exponentially

- Preimage security, e.g., MMO-AES128 *64-bit*
- Collision security, SHAKE128 (Part of SHA-3) *$256/3 = 85.3$ bit*
- Key recovery security, AES-128 *$128/2 = 64$ - bit*

New block cipher designs with larger block size / digest size are needed.

Modes need to be re-designed immediately !

Quantum Algorithm to the Order

The problem: given a , N with $\gcd(a, N) = 1$, find the smallest integer r , such that $a^r = 1 \bmod N$

Algorithm: let $f(x) = a^x \bmod N$, then $f(x)$ is a periodic function with period r , and this can be done in polynomial time.

$$f(x+r) = a^{x+r} = a^x \cdot a^r = a^x = f(x)$$

Classic Factoring Algorithm

Goal: find factors of N (big)

r is even

$$r = 2 \cdot K, K \in \mathbb{Z}$$

$$a^r = 1 \pmod{N}$$

$$a^{2K} - 1 = 0$$

$$(a^K + 1)(a^K - 1) = 0$$

1. Pick a random number $1 < a < N$.
2. Compute $K = \gcd(a, N)$, the **greatest common divisor** of a and N .
3. If $K \neq 1$, then K is a **nontrivial** factor of N , with the other factor being $\frac{N}{K}$ and we are done.
4. Otherwise, use the quantum subroutine to find the order r of a . $a^r = 1 \pmod{N}$
5. If r is odd, then go back to step 1.
6. Compute $g = \gcd(N, a^{r/2} + 1)$. If g is nontrivial, the other factor is $\frac{N}{g}$, and we're done. Otherwise, go back to step 1.

Quantum Algorithm for Discrete Log

The problem: Given a , N and generator g for Z_N^* , and y , find x , such that $g^x = y \bmod N$.

$$x = \text{Dlog}_a y \bmod N$$

Algorithm: construct $f(x_1, x_2) = g^{x_1} y^{x_2} \bmod N$, find the period (s_1, s_2) of the function by quantum algorithm, then $x = (-s_1)s_2^{-1} \bmod (\phi(N))$

$$g^{s_1 + x_1} \cdot y^{s_2 + x_2} = g^{x_1} \cdot y^{x_2} \Rightarrow g^{s_1} \cdot y^{s_2} = 1 \Rightarrow g^{s_1} \cdot g^{x \cdot s_2} = g^0$$

$$x = \frac{-s_1}{s_2} \Leftrightarrow s_1 + x \cdot s_2 = 0 \Leftrightarrow g^{s_1 + x \cdot s_2} = g^0$$

Outline: Post-Quantum Cryptography

I. Alternative hard problems

v.s. factoring $\rightarrow$ Discrete log.

- Lattices
- Multivariate Quadratic Problems
- One-way Hash functions

II. PQC Standards

- NIST PQC Standardization Process
- Signatures: CRYSTALS-Dilithium, SPHINCS+
- KEM: CRYSTALS-Kyber

Meet The Quantum Adversaries

- Grover's Search

- Quadratic speedup on brute force search of preimage: $2^n \rightarrow 2^{n/2}$
- Speedup on collision finding: $2^{n/2} \rightarrow 2^{n/3}$

Example: for hash function standard AES-MMO, a quantum computer can find a preimage in 2^{64} computations instead of 2^{128} and a collision in 2^{43} computations instead of 2^{64}

- Simon's Period Finding

- Linearized time for finding period behaviors, i.e., if $f(x) = f(x+s)$ for all x , then finding the period s is **easy** by quantum computer
- Break MAC standards (CBC-MAC, PMAC, GMAC, etc.)

- Quantum Fourier Transform

- An efficient quantum alternative for Fast Fourier Transform
- Accelerate key recovery attacks

Post-Quantum Cipher (PQC)



Threats on Symmetric-key Ciphers

- Grover's search provides a quadratic speedup on unstructured search
- Needs key length at least 256 bits
- AES128/192 are vulnerable
- The post quantum cipher discussed later focuses on asymmetric-key ciphers

Post Quantum Cipher

- Ciphers based on the hardness of factoring or discrete logarithm are **vulnerable** to quantum adversaries
- Post Quantum Cipher (PQC) is designed based on alternative hard math problems
 - Lattice based encryptions and signatures: security relies on hardness of finding short vectors in some lattices.
 - Coding based encryptions: security relies on hardness of decoding error-correcting codes.
 - Multivariate Quadratic signatures: Security relies on hardness of solving systems of multivariate equations over finite fields.
 - Hash-based signature: require only a secure hash function (hard to find second preimages).

Intuition to Lattices: An Easy Problem

$$\begin{pmatrix} \mathbf{A} \end{pmatrix} \begin{pmatrix} \mathbf{y} \end{pmatrix} = \begin{pmatrix} \mathbf{z} \end{pmatrix} \bmod p$$

$$A^{-1} A \cdot \vec{y} = A^{-1} \vec{z} \Rightarrow \vec{y} = A^{-1} \cdot \vec{z}$$

- Given (A, z) and p, find y
- Solution: compute the inverse of A then multiply z

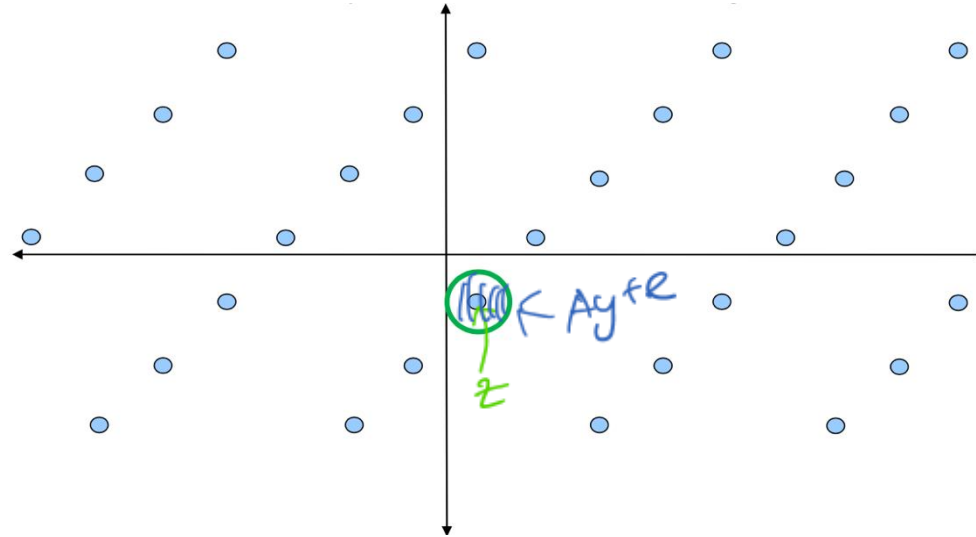
Hard Alternative

$$A \cdot \vec{y} + \vec{e} = \vec{z}, \text{ given } (A, \vec{z}) \\ \text{hard to find } (\vec{y}, \vec{e})$$

$$\begin{pmatrix} \mathbf{A} \end{pmatrix} \begin{pmatrix} \mathbf{y} \end{pmatrix} + \begin{pmatrix} \mathbf{e} \end{pmatrix} = \begin{pmatrix} \mathbf{z} \end{pmatrix} \bmod p$$

- Entries of y and e are small in magnitude to enforce uniqueness
- Given (A, z) and p , find (y, e) with a small magnitude
- Hard to solve!

Lattice Problem



- The solution space of $Ay + e = z \pmod{p}$ forms a lattice
- Lattice problem: find the point closest to the origin (Short Vector Problem SVP)

Multivariate Quadratic (MQ)

$$p^{(1)}(w_1, \dots, w_n) = \sum_{i=1}^n \sum_{j=i}^n p_{ij}^{(1)} \cdot w_i w_j + \sum_{i=1}^n p_i^{(1)} \cdot w_i (+p_0^{(1)})$$

$$p^{(2)}(w_1, \dots, w_n) = \sum_{i=1}^n \sum_{j=i}^n p_{ij}^{(2)} \cdot w_i w_j + \sum_{i=1}^n p_i^{(2)} \cdot w_i (+p_0^{(2)})$$

$\vdots$

$$p^{(m)}(w_1, \dots, w_n) = \sum_{i=1}^n \sum_{j=i}^n p_{ij}^{(m)} \cdot w_i w_j + \sum_{i=1}^n p_i^{(m)} \cdot w_i (+p_0^{(m)})$$

- They consists of quadratic terms $w_i w_j$ and linear terms w_i defined over modulo 2
- Each variable w_i can be 0 or 1
- These discrete equations are hard to solve (unlike their counterpart on real numbers)

Security

The security of multivariate schemes is based on the MQ problem:

Problem MQ: Given m multivariate quadratic polynomials $p^{(1)}, \dots, p^{(m)}$, find a vector $\mathbf{w} = (w_1, \dots, w_n)$ such that $p^{(1)}(\mathbf{w}) = \dots = p^{(m)}(\mathbf{w}) = 0$.

- Classic: NP hard
- Quantum: Hard on average

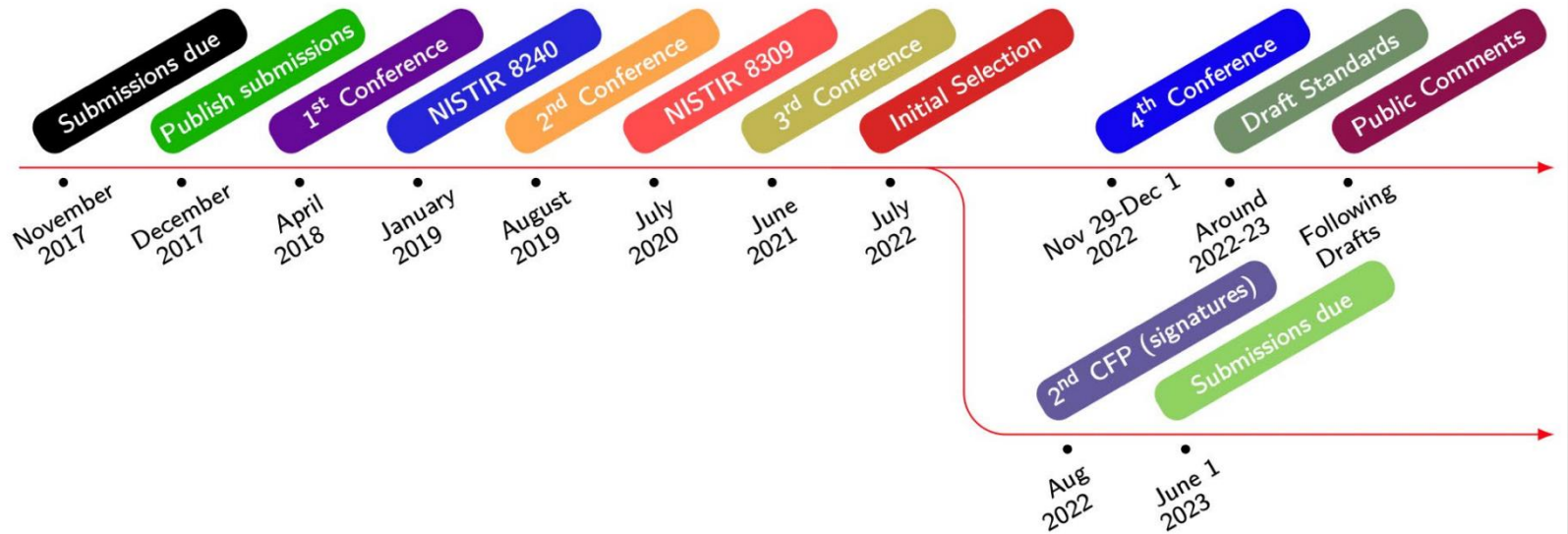
Hash-based Signatures

- Security relies only on the security of cryptographic hash function
- What we know so far:
 - Preimage Resistance: reduced from 2^n to $2^{n/2}$
 - Collision Resistance: reduced from $2^{n/2}$ to $2^{n/3}$
- Hence, hash functions are still considered **secure** when n is large enough, e.g., SHA-384, SHA-512, SHA3-384, and SHA3-512.
Handwritten notes: SHA-2, $384/3 = 128$, $512/3 = 170.6$
- Hence, no plan for new hash function schemes to resist quantum adversaries.

Post-Quantum Cryptography Standards



NIST PQC Standardization Process



Security Criteria

Secure against both classical and quantum attacks

Level	Security Description		
I	At least as hard to break as AES128	(exhaustive key search)	64-bit
II	At least as hard to break as SHA256	(collision search)	85-bit
III	At least as hard to break as AES192	(exhaustive key search)	96-bit
IV	At least as hard to break as SHA384	(collision search)	128-bit
V	At least as hard to break as AES256	(exhaustive key search)	128-bit

3rd Round Selection for Standardization

- KEM
 - CRYSTALS-Kyber
- Signatures
 - CRYSTALS-Dilithium
 - Falcon
 - SPHINCS+

CRYSTALS-Dilithium

- Recommended by NIST as the **PRIMARY SIGNATURE ALGORITHM USED**
- Public key size 1.5 KB, signature size 2.7 KB
- Design based on “Fiat-Shamir with Aborts” technique
 - Rejection sampling is used to sample signatures that do not reveal secret information
- Signature compression (> 50% smaller)
- New: Compression of public key (60% smaller, 100 byte larger signature)
- New: Hardness based on Module-LWE/SIS
- New: Very efficient implementation

Main Operations

- Evaluations of SHAKE256 (can use another XOF too)
 - Independent sampling of polynomials: Allows for parallel use of SHAKE
- Operations in the polynomial ring $R = (\mathbb{Z}_p[x])/x^{256} + 1$
 - The ring with form $(\mathbb{Z}_p[x])/x^n + 1$, for a prime p and dimension n , has been the most widely-used ring in the literature
 - Dimension n is chosen as 256, which is large enough to efficiently encrypt 256-bit keys and to a good security bound
 - Prime p is chosen as: $2^{23} \cdot 2^{13} + 1$

Both operations are very efficient!

Signatures with Uniform Sampling (SUS)

Key generation:

$$\mathbf{A} \leftarrow R^{5 \times 4}$$

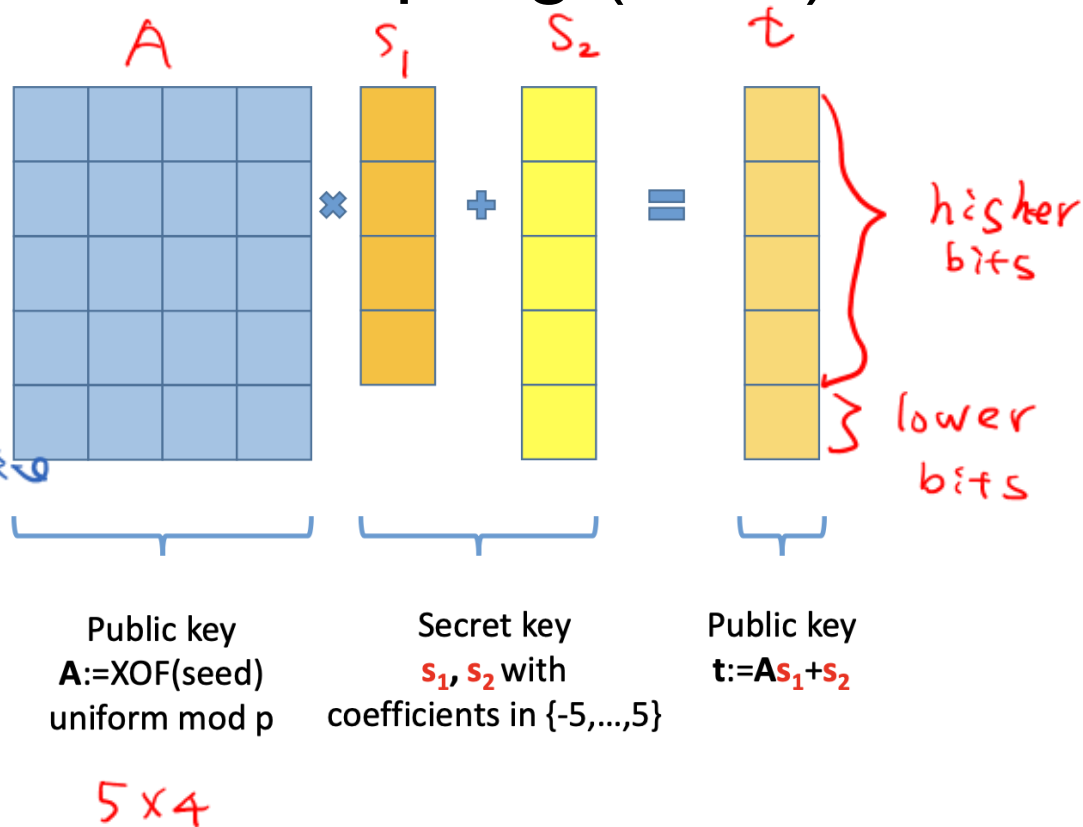
$$\mathbf{s}_1 \leftarrow S_5^4, \mathbf{s}_2 \leftarrow S_5^5$$

$$\mathbf{t} = \mathbf{A}\mathbf{s}_1 + \mathbf{s}_2$$

$$pk = (\mathbf{A}, \mathbf{t}), sk = (\mathbf{A}, \mathbf{t}, \mathbf{s}_1, \mathbf{s}_2)$$

given the hardness of lattice problem, given $\mathbf{A}$ and $\mathbf{t}$, it is hard to find $\mathbf{s}_1, \mathbf{s}_2$.

- Public key: $\mathbf{A}, \mathbf{t}$
- Secret key: $\mathbf{s}_1, \mathbf{s}_2$



SUS - Signing and Verification

Signing:

$$y \leftarrow S_\gamma^4$$

$$w = Ay$$

$$c = H(\text{High}(w), M) \in B_{60}$$

$$z = y + cs_1$$

If $\|z\|_\infty > \gamma - \beta$ or $\|\text{Low}(w - cs_2)\|_\infty > \gamma - \beta$, restart

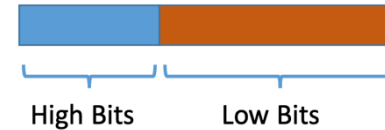
$\text{sig} = (z, c)$ ($\|z\|_\infty \leq \gamma - \beta$ and

$$\|\text{Low}(w - cs_2)\|_\infty \leq \gamma - \beta$$

Verification:

$$c' = H(\text{High}(Az - ct), M)$$

If $\|z\|_\infty \leq \gamma - \beta$ and $c' = c$, accept



$$\text{High}(w - cs_1) = \text{High}(w)$$

$$\begin{aligned} Az - ct &= A \cdot (y + cs_1) - ct \\ &= Ay + Acs_1 - ct \\ &= w + c \cdot (AS_1 - t) \\ &= w - cs_2 \end{aligned}$$

- Function high/low takes the high/low bits for a bitstring
- H stands for a hash function
- M stands for the message

SUS – Some Properties

If $\|\mathbf{z}\|_{\infty} > \gamma - \beta$ or $\|\text{Low}(\mathbf{w} - c\mathbf{s}_2)\|_{\infty} > \gamma - \beta$, restart

- The check is needed for security

$$\|\text{Low}(\mathbf{w} - c\mathbf{s}_2)\|_{\infty} > \gamma - \beta$$

- This check is needed for correctness
- $\beta = \max(|c\mathbf{s}_2|)$, when $|\text{low}(\mathbf{A}\mathbf{y} - c\mathbf{s}_2)| \leq \gamma - \beta$, $\text{high}(\mathbf{A}\mathbf{y} - c\mathbf{s}_2) = \text{high}(\mathbf{A}\mathbf{y})$

$$c' = H(\text{High}(\mathbf{A}\mathbf{z} - c\mathbf{t}), M)$$

- We have $\mathbf{A}\mathbf{z} - c\mathbf{t} = \mathbf{A}\mathbf{y} - c\mathbf{s}_2$

Dilithium: Public Key Compression

$$As_1 + s_2 = t_0 + bt_1$$

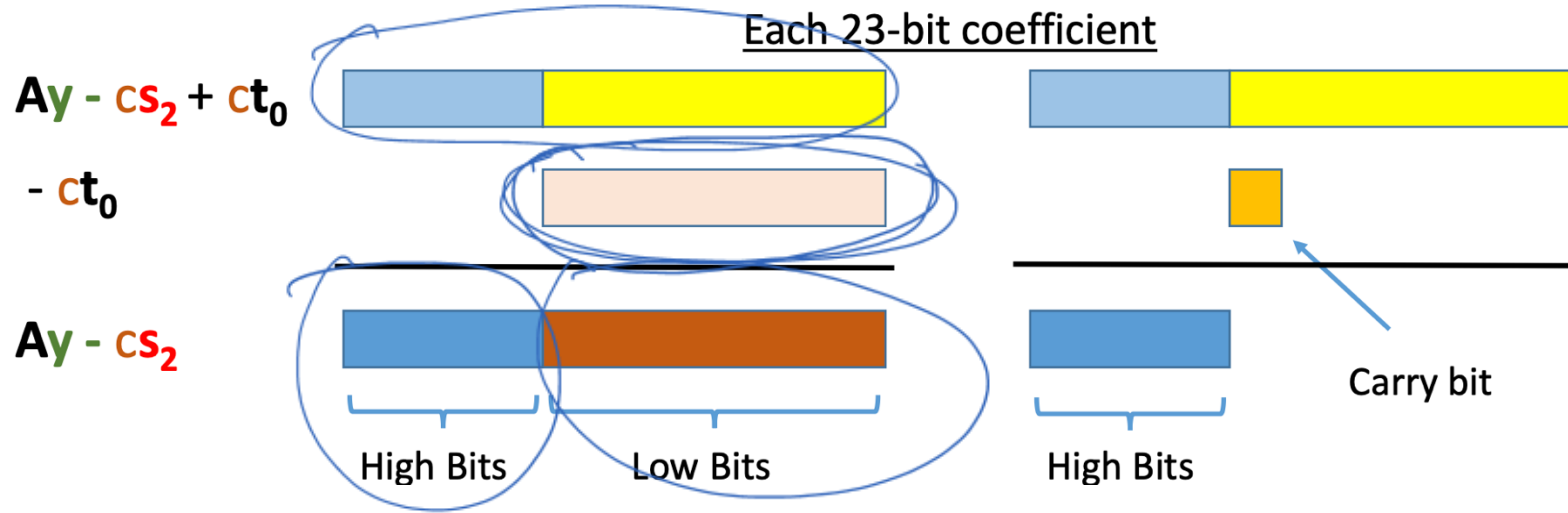
$$t = \boxed{t_0} + b \cdot t_1$$

↓

Low part

- Replace t in public key by t_1 (23 bits to 9 bits)
- In verification, we still need to compute $\text{high}(Az - ct)$, but with key compression we can only compute $\text{high}(Az - cbt_1)$
- Dilithium includes a **carry hint vector h** in the signature to correct $\text{high}(Az - cbt_1)$ to $\text{high}(Az - ct)$ - $\text{high}(Az - cbt_1) = 0/1$
- The hint h adds 100-200 bytes to the signature, while saves approximately 2KB in the public key

The Carry Hint Vector



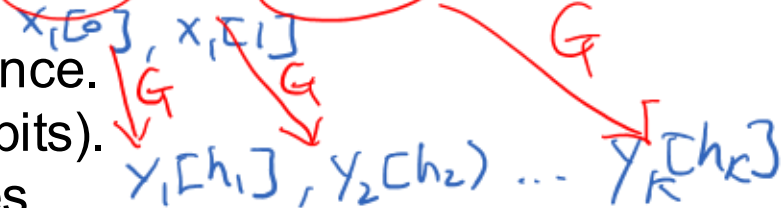
- By including the carry bit hint vector into the signature, we can successfully recover $\text{high}(Az - ct)$ from $\text{high}(Az - cbt_1)$.

SPHINCS+

- An advanced version of SPHINCS (Eurocrypt 2015)
- Mainly reduce the signature size of SPHINCS
- A stateless hash-based signature scheme
 - What is hash-based signature scheme?
 - What does stateless mean?
 - What is SPHINCS?
 - What does SPHINCS+ improve?

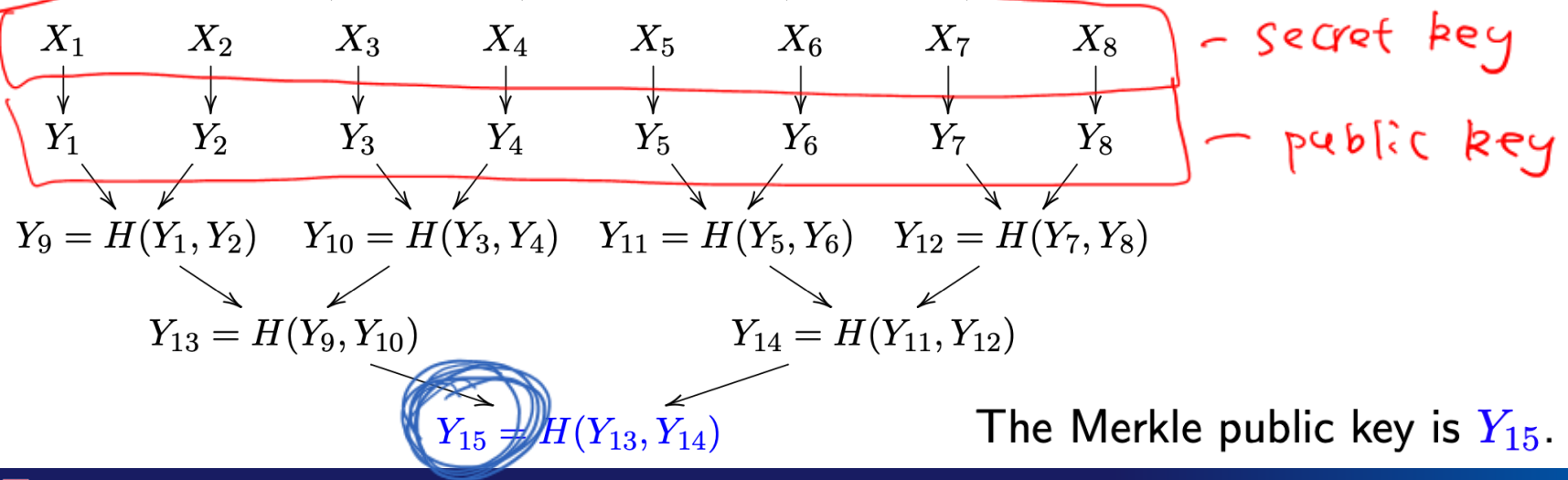
What is hash-based signature scheme?

- 1979 Lamport **one-time signature scheme**.
- Fix a k -bit one-way function $G : \{0, 1\}^k \rightarrow \{0, 1\}^k$ and hash function $H : \{0, 1\}^* \rightarrow \{0, 1\}^k$.
- Signer's secret key X : $2k$ strings $x_1[0], x_1[1], \dots, x_k[0], x_k[1]$, each k bits. Total: $2k^2$ bits.
- Signer's public key Y : $2k$ strings $y_1[0], y_1[1], \dots, y_k[0], y_k[1]$, each k bits, computed as $y_i[b] = G(x_i[b])$. Total: $2k^2$ bits.
- Signature $S(X, r, m)$ of a message m : $r, x_1[h_1], \dots, x_k[h_k]$ where $H(r, m) = (h_1, \dots, h_k)$.
- Must never use secret key more than once.
- Usually choose $G = H$ (restricted to k bits).
- 1979 Merkle extends to more signatures.

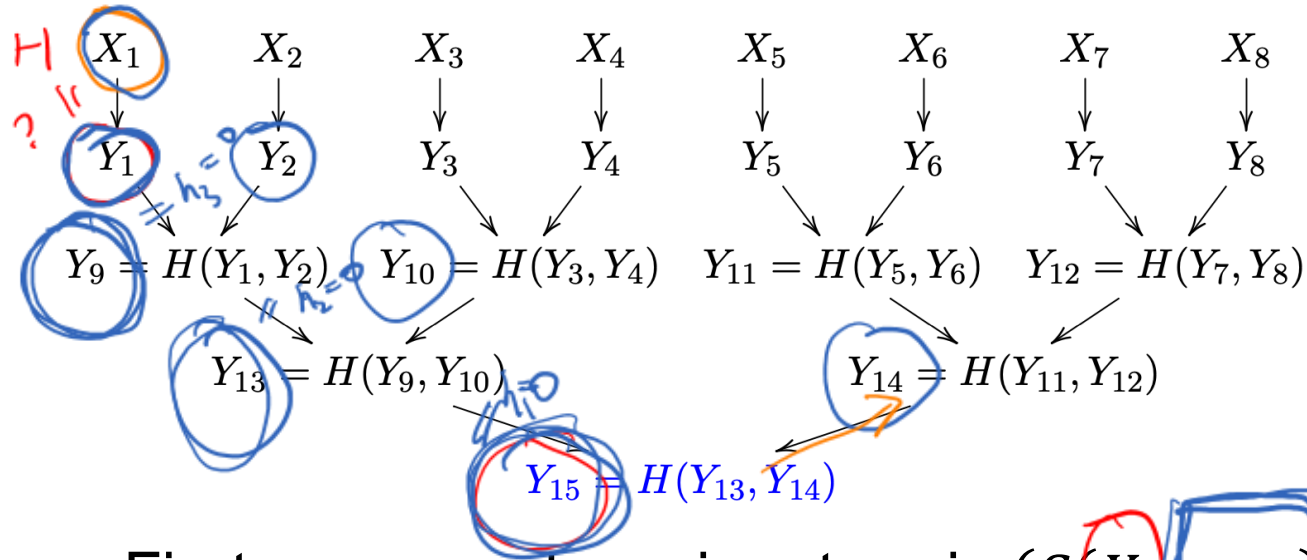


What is hash-based signature scheme?

- **Eight Lamport one-time keys**
- $Y_1, Y_2, \dots, Y_8$ with corresponding $X_1, X_2, \dots, X_8$,
 $X_i = (x_{i,1}[0], x_{i,1}[1], \dots, x_{i,k}[0], x_{i,k}[1])$



What is hash-based signature scheme?



- First message has signature is $(S(X_1, r, m), Y_1, Y_2, Y_{10}, Y_{14})$.
- Verify: Check signature $S(X_1, r, m)$ on m against Y_1
- $Y_{15} = H(H(H(Y_1, Y_2), Y_{10}), Y_{14})$

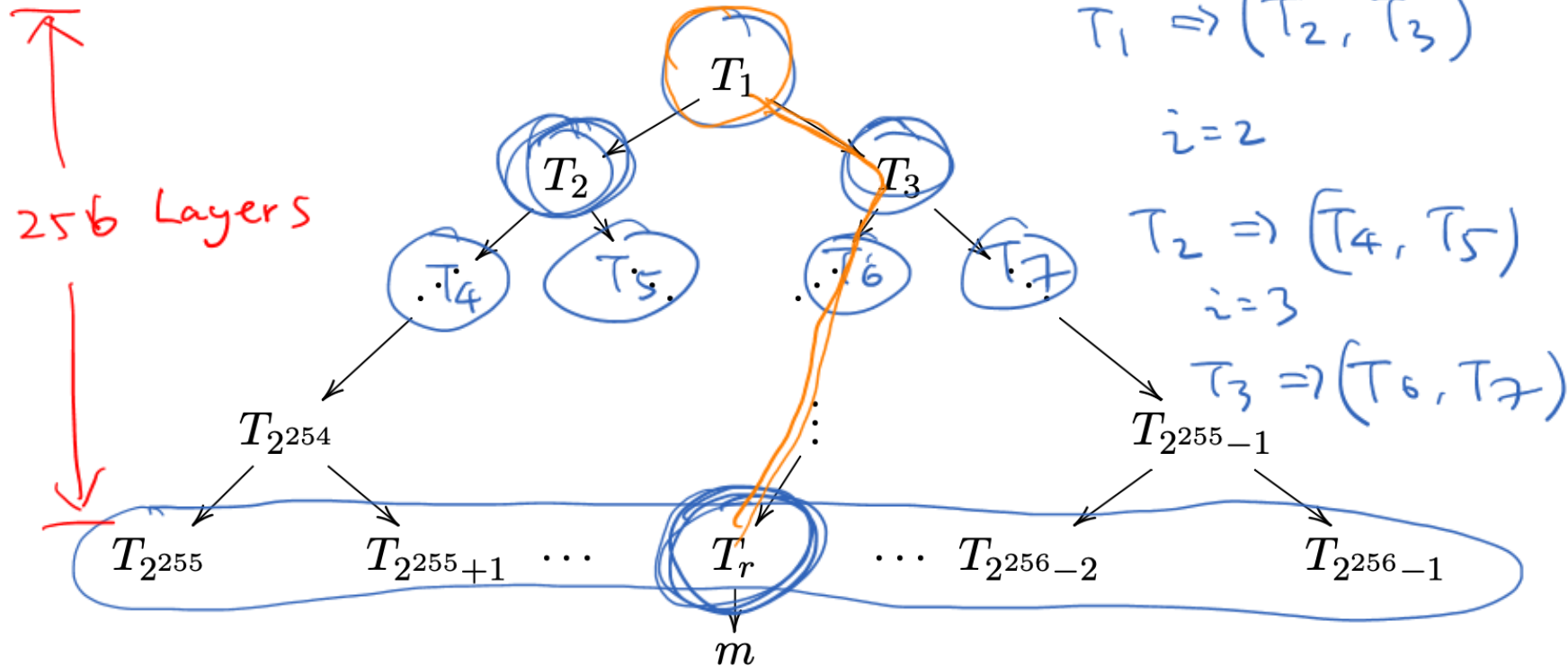
What is hash-based signature scheme?

- Pros:
 - Post quantum
 - Only need secure hash function
 - Small public key
 - Security well understood
 - Fast
- Cons:
 - Biggish signature and secret key
 - Stateful

Become Stateless

- Use Huge Tree to avoid collision
- Signer chooses random $r \in \{2^{255}, 2^{255} + 1, \dots, 2^{256} - 1\}$
- Uses one-time public key T_r to sign message
- Uses one-time public key T_i to sign (T_{2i}, T_{2i+1}) for $i < 2^{255}$
- Generates i -th secret key as $H_k(i)$ where k is master secret.

Become Stateless



Become Stateless

- Problem:
 - Signatures are painfully long
- Examples:
 - Debian operating system is designed for frequent upgrades. At least one new signature for each upgrade. Typical upgrade: one package or just a few packages. 1.2 MB average package size. 0.08 MB median package size.
 - HTTPS typically sends multiple signatures per page. 1.8 MB average web page in Alexa Top 1000000

SPHINCS-256

$$2\text{GHz} = 2^3 \text{ cycles per s} \\ = 2\text{B}$$

$$\frac{51\text{M}}{2\text{B}} = 0.0255 \text{ seconds}$$

- Greatly reduce size

- 0.041 MB signature 41 KB
- 0.001 MB public key 1 KB
- 0.001 MB private key 1 KB

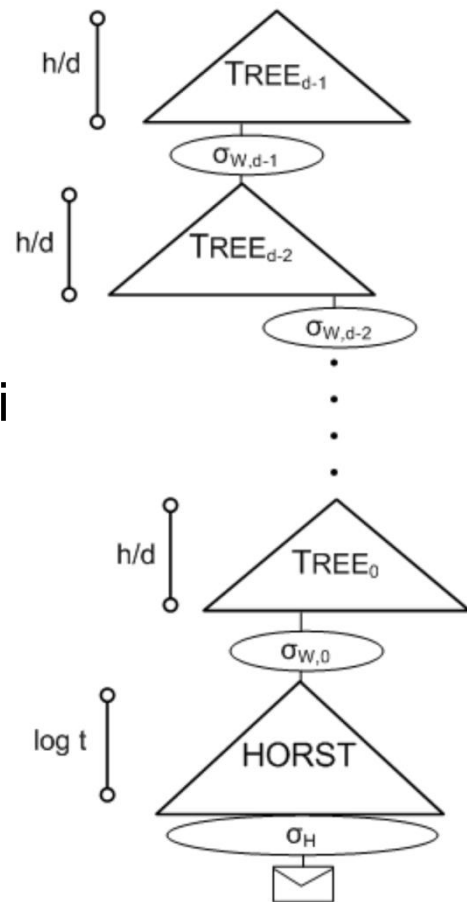
- Good Speed

- 51.1 million cycles to sign. (RSA-3072: 14.2 million.) 3.5x slower
- 1.5 million cycles to verify. (RSA-3072: 0.1 million.) 15x slower
- 3.2 million cycles for keygen. (RSA-3072: 950 million.) 300x faster

- Designed for 2^{128} post-quantum security

SPHINCS-256

- Main improvement
 - Select index pseudorandomly
 - Use a few-time signature key-pair on leaves to si
 - Few index collisions allowed
 - Allows to reduce tree height
 - Use hypertree: then $d \ll h$



SPHINCS+

- Further reduce size
- Changes
 - Allow for 264 instead of 250 signatures per key pair
 - Add multi-target attack mitigation (Tweakable hash functions)
 - Tree-less WOTS+ public key compression
 - New few-time signature scheme FORS
 - Verifiable index selection
 - Optional non-deterministic signatures
 - Simple instantiations with SHAKE256, SHA-256, Haraka

<https://sphincs.org/>



SPHINCS+

- Further reduce size
- New features
 - Multi-target attack protection
 - Tree-less WOTS+ public key compression
 - FORS
 - Verifiable index selection

<https://sphincs.org/>



CRYSTALS-Kyber

- Construct based on **Module-LWE**(Learning With Errors) Problems. This is a problem in lattice cryptography where given a random linear equation with some small error(centered binomial noise), it's hard to solve for the unknowns.

$$\mathcal{R} = \mathbb{Z}_q[X]/(X^{256} + 1) \text{ with } q = 3329$$

~2¹²

- Key Encapsulation Mechanism (KEM): In Kyber, a KEM is used to secure the transmission of keys. A sender encapsulates a key into a ciphertext using the recipient's public key, and the recipient decapsulates it back into the original key using their private key.

CRYSTALS-Kyber is fast

cpu
register

64-bit

AVX 128-bit

AVX2 256-bit

AVX512 512-bit

Kyber512 ($k = 2$, level 1)

Sizes (in Bytes)

sk:	1632
pk:	800
ct:	736

Haswell Cycles (AVX2)

gen:	29100
enc:	46196
dec:	39410

256-bits
cycles

Kyber768 ($k = 3$, level 3)

Sizes (in Bytes)

sk:	2400
pk:	1184
ct:	1088

Haswell Cycles (AVX2)

gen:	57340
enc:	78692
dec:	68620

Kyber1024 ($k = 4$, level 5)

Sizes (in Bytes)

sk:	3168
pk:	1568
ct:	1568

Haswell Cycles (AVX2)

gen:	81244
enc:	109584
dec:	97280



CRYSTALS-Kyber is fast and small

Kyber512 ($k = 2$, level 1)			
Stack usage (in Bytes)		Cortex-M4	Cycles
gen:	2952	gen:	513992
enc:	2552	enc:	652470
dec:	2560	dec:	620946
Kyber768 ($k = 3$, level 3)			
Stack usage (in Bytes)		Cortex-M4	Cycles
gen:	3848	gen:	976205
enc:	3128	enc:	1146021
dec:	3072	dec:	1094314
Kyber1024 ($k = 4$, level 5)			
Stack usage (in Bytes)		Cortex-M4	Cycles
gen:	4360	gen:	1574351
enc:	3584	enc:	1779192
dec:	3592	dec:	1708692

32-bit cpu